

Cricket App

AWS EC2 Deployment Guide

Complete Documentation for Deploying on AWS Free Tier

Domain: <https://cricketapp.in>

EC2 Instance: 13.50.248.161 (t2.micro - Free Tier)

OS: Ubuntu 22.04 LTS

Date: August 17, 2026

Table of Contents

1. Project Overview
2. Architecture Diagram
3. Technology Stack
4. AWS Setup (Step-by-Step)
 - 4.1 AWS Account Creation
 - 4.2 EC2 Instance Launch
 - 4.3 Security Groups
 - 4.4 DNS Configuration (GoDaddy)
5. Server Setup
 - 5.1 SSH Connection
 - 5.2 Docker Installation
 - 5.3 Repository Clone
6. Deployment
 - 6.1 Environment Variables (.env)
 - 6.2 Docker Compose Production
 - 6.3 Building & Starting Containers
7. SSL Certificate (Let's Encrypt)
8. API Endpoints
9. How Each Component Works
10. Backup & Maintenance
11. Cost Breakdown (Free Tier)
12. Troubleshooting

1. Project Overview

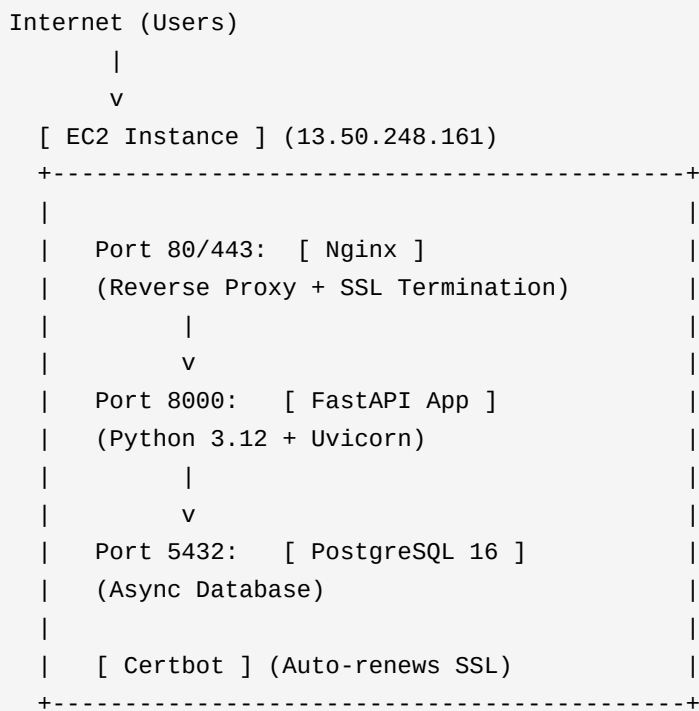
The Cricket App is a RESTful API built with FastAPI (Python) that manages cricket teams, players, locations, and OTP-based phone verification. It is designed as a backend service for a cricket scoring/management application.

Key Features:

- Team Management - Create, update, list cricket teams
- Player Management - Register players, manage profiles
- Location Data - Countries, states, and cities database (seeded automatically)
- OTP Verification - Phone number verification via 2factor.in SMS API
- File Uploads - Profile photos and team logos stored locally
- Health Check - /health endpoint for monitoring

2. Architecture Diagram

The production deployment uses 4 Docker containers orchestrated by Docker Compose:



3. Technology Stack

Component	Technology	Purpose
Language	Python 3.12	Backend API runtime
Framework	FastAPI	Async web framework
Server	Uvicorn	ASGI server
Database	PostgreSQL 16	Production database
ORM	SQLAlchemy 2.0	Async database ORM
Migrations	Alembic	Database schema migrations
Reverse Proxy	Nginx 1.27	SSL, load balancing, routing
SSL	Let's Encrypt	Free HTTPS certificates
Containers	Docker + Compose	Isolated deployment
Cloud	AWS EC2 t2.micro	Free tier hosting
DNS	GoDaddy	Domain management
SMS/OTP	2factor.in	Phone verification

4. AWS Setup (Step-by-Step)

4.1 AWS Account Creation

- Go to <https://aws.amazon.com> and click 'Create an AWS Account'
- Enter email, password, and payment info (credit card required for verification)
- Select 'Basic Support - Free' plan
- Complete identity verification (OTP on phone)

4.2 EC2 Instance Launch

From the EC2 Dashboard, click 'Launch Instance' with these settings:

Setting	Value
Name	cricketapp
AMI (OS)	Ubuntu 22.04 LTS
Instance Type	t2.micro (Free Tier eligible)
Key Pair	cricketapp-key.pem (downloaded)
Storage	30 GB gp2 (EBS)

4.3 Security Groups (Firewall Rules)

Inbound rules configured on the EC2 instance:

Type	Port	Source	Description
SSH	22	Your IP	Remote server access
HTTP	80	0.0.0.0/0	Web traffic + SSL verification
HTTPS	443	0.0.0.0/0	Secure web traffic

4.4 DNS Configuration (GoDaddy)

In GoDaddy DNS Management, an A record was created to point the domain to the EC2 IP:

Type	Name	Value	TTL
A	@	13.50.248.161	600 seconds

This means: when anyone visits cricketapp.in, DNS resolves to the EC2 server's IP address.

5. Server Setup

5.1 SSH Connection

To connect to the EC2 instance from a local machine:

```
ssh -i "cricket.pem" ubuntu@13.50.248.161
```

The .pem file is a private key that proves your identity to the server. Without it, nobody can access the server via SSH.

5.2 Docker Installation

On the Ubuntu server, these commands were executed:

```
sudo apt update && sudo apt upgrade -y  
sudo apt install -y docker.io docker-compose-v2 git  
sudo usermod -aG docker ubuntu
```

- docker.io - The Docker engine that runs containers
- docker-compose-v2 - Tool to manage multi-container apps
- git - To clone the source code from GitHub

5.3 Repository Clone

```
git clone https://github.com/sanjithhithub/cricketapp.git  
/home/ubuntu/cricketapp
```

6. Deployment

6.1 Environment Variables (.env)

A production .env file was created with these values:

```

DATABASE_URL=postgresql+asyncpg://cricket:CricketDB2026@cricketdb:5432/cricketdb
POSTGRES_PASSWORD=CricketDB2026
TWO_FACTOR_API_KEY=40cda3df-77a2-11f1-803e-0200cd936042
TWO_FACTOR_OTP_TEMPLATE=
DOMAIN=cricketapp.in

```

What each variable does:

- DATABASE_URL - Connection string for the app to reach PostgreSQL inside Docker
- POSTGRES_PASSWORD - Password for the PostgreSQL database user
- TWO_FACTOR_API_KEY - API key for 2factor.in SMS OTP service
- DOMAIN - The public domain name for nginx and SSL

6.2 Docker Compose Production (docker-compose.prod.yml)

The production Docker Compose file defines 4 services that work together:

Service 1 - cricketapp (FastAPI):

- Builds from the Dockerfile (Python 3.12-slim)
- Runs alembic migrations on startup
- Starts Uvicorn server on port 8000
- Health check: GET /health every 30s

Service 2 - cricketdb (PostgreSQL):

- Uses postgres:16-alpine (lightweight)
- Stores data in a Docker volume 'pgdata'
- Health check: pg_isready every 5s

Service 3 - nginx (Reverse Proxy):

- Listens on ports 80 (HTTP) and 443 (HTTPS)
- Routes all traffic to the FastAPI app
- Handles SSL termination

Service 4 - certbot (SSL Renewal):

- Automatically renews the SSL certificate
- Runs every 12 hours in the background

6.3 Building & Starting Containers

```

cd /home/ubuntu/cricketapp
sed 's/___DOMAIN___/cricketapp.in/g' \
    deploy/nginx-http.conf > deploy/nginx.conf
docker compose -f docker-compose.prod.yml up -d --build

```

Cricket App - AWS EC2 Deployment Guide

This command: (1) generates the nginx config with the correct domain, (2) builds the Docker image from the Dockerfile, (3) starts all 4 containers in detached mode.

7. SSL Certificate (Let's Encrypt)

SSL (HTTPS) encrypts all traffic between users and the server. We used Let's Encrypt, a free certificate authority.

The init-ssl.sh script performed these steps:

- Step 1: Started nginx in HTTP-only mode for domain verification
- Step 2: Certbot proved domain ownership via ACME challenge (/well-known/acme-challenge/)
- Step 3: Let's Encrypt issued a free SSL certificate
- Step 4: Nginx was switched to HTTPS mode with the new certificate
- Step 5: Certificate auto-renews every 12 hours via certbot container

```
bash deploy/init-ssl.sh cricketapp.in
```

```
# Result:
# Certificate saved at: /etc/letsencrypt/live/cricketapp.in/fullchain.pem
# Key saved at: /etc/letsencrypt/live/cricketapp.in/privkey.pem
# Expires: 2026-11-15 (auto-renews before expiry)
```

8. API Endpoints

Endpoint	Method	Description
/health	GET	Health check - returns {"status": "ok"}
/locations	GET	List all countries, states, and cities
/country-codes	GET	List all country calling codes
/verify-otp	POST	Verify phone OTP for registration
/players/...	GET/POST	Player CRUD operations
/teams/...	GET/POST	Team CRUD operations
/uploads/...	GET	Static file serving (images)
/docs	GET	Swagger API documentation

9. How Each Component Works

FastAPI Application (app/main.py)

When the app starts, it:

1. Creates database tables via SQLAlchemy Base.metadata.create_all()
2. Seeds location data (countries, states, cities) into PostgreSQL
3. Starts Uvicorn ASGI server on 0.0.0.0:8000
4. Registers route handlers for /locations, /verify-otp, /health, etc.
5. Mounts /uploads as a static file directory

Database (app/database.py)

The database module reads DATABASE_URL from environment variables.

It creates an async SQLAlchemy engine and session factory.

For PostgreSQL, it uses asyncpg as the async driver.

The get_db() function provides a session to each request handler.

Nginx (Reverse Proxy)

Nginx sits in front of the FastAPI app and:

- Terminates SSL (decrypts HTTPS traffic)
- Forwards requests to the app container on port 8000
- Adds security headers (X-Real-IP, X-Forwarded-For)
- Limits upload size to 10MB
- Redirects HTTP to HTTPS

Docker Entrypoint (docker-entrypoint.sh)

Before the app starts, the entrypoint script runs:

1. alembic upgrade head (applies all database migrations)
2. uvicorn app.main:app --host 0.0.0.0 --port 8000

10. Backup & Maintenance

Automated Database Backups

The backup script (deploy/backup.sh) runs nightly via cron:

```
# Schedule backup at 3 AM daily:
crontab -e
# Add this line:
0 3 * * * /home/ubuntu/cricketapp/deploy/backup.sh >> /home/ubuntu/backups/cron.log 2>&1
```

What it does:

- Dumps PostgreSQL database to a compressed .sql.gz file
- Saves to /home/ubuntu/backups/
- Keeps only the last 7 days of backups
- Filename format: cricketdb-YYYY-MM-DD.sql.gz

Updating the Application

When you push new code to GitHub, deploy with:

```
ssh -i cricket.pem ubuntu@13.50.248.161
cd /home/ubuntu/cricketapp
git pull
docker compose -f docker-compose.prod.yml up -d --build
```

Viewing Logs

```
# App logs:
sudo docker logs cricketapp --tail 50

# Nginx logs:
sudo docker logs cricketapp-nginx --tail 50

# Database logs:
sudo docker logs cricketdb --tail 50
```

Restarting Services

```
# Restart everything:
docker compose -f docker-compose.prod.yml restart

# Restart just the app:
docker compose -f docker-compose.prod.yml restart cricketapp
```

11. Cost Breakdown (AWS Free Tier)

Resource	Specification	Free Tier Limit	Monthly Cost
EC2 Instance	t2.micro, 1 vCPU, 1GB RAM	750 hrs/month	\$0.00

Cricket App - AWS EC2 Deployment Guide

EBS Storage	30 GB gp2 SSD	30 GB/month	\$0.00
Data Transfer	Outbound	100 GB/month	\$0.00
		Total:	\$0.00

Note: Free tier is available for 12 months from account creation. After that, t2.micro costs approximately \$0.0104/hour (~\$7.50/month). Always check AWS Free Tier dashboard to avoid unexpected charges.

12. Troubleshooting

App won't start - Database connection error

- Check DATABASE_URL in .env for special characters in password
- Ensure postgres container is healthy: `docker compose ps`
- Check logs: `docker logs cricketapp`

Port 80/443 already in use

```
sudo lsof -i :80
sudo systemctl stop apache2 # or nginx
```

SSL certificate not working

- Ensure DNS A record points to EC2 public IP
- Ensure ports 80 and 443 are open in AWS Security Group
- Run: `bash deploy/init-ssl.sh cricketapp.in`

Cannot SSH into EC2

- Check .pem file has correct permissions
- Verify Security Group allows SSH (port 22) from your IP
- Make sure you're using 'ubuntu' as the username

Container keeps restarting

```
sudo docker compose -f docker-compose.prod.yml logs cricketapp
# Look for error messages in the output
```

Deployment Complete!

API URL: <https://cricketapp.in>

Swagger Docs: <https://cricketapp.in/docs>

Health Check: <https://cricketapp.in/health>